

PHPの「型」は どこまで戦えるか

Can PHP Types Still Compete?
Pushing the Limits of Static Analysis.



pixiv Inc.
USAMI Kenta

pixiv

みなさまは
PHPの無限の可能性が
好きですか？

問題です

PHPの型についての
質問で正しいものは？
(全4問)

1. PHPは 型なし言語である

2. PHPには 型宣言がある

3. PHPに書く型は 実行時に保証されない

4. PHPは 静的型付きである

正しいものは？（複数回答可能）

1. PHPは型なし言語である
2. PHPには型宣言がある
3. PHPに書く型は実行時に保証されない
4. PHPは静的型付きである

お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE / にゃんだーすわん
- ピクシブ株式会社 Platform Div > WebTechnology Team PHPer
 - 2012年末から現職、APIとかCIとかいろいろなところを見つめてきました
 - 最近チームが再編されてインフラっぽい仕事もしてます
- Emacs PHP Modeを開発しています (2017年-)
- プログラミング言語にちょっとこだわりのある素人 (spcamp2010)

さて

型がついている
#とは

型推論規則に従って
すべての項に矛盾なく
型が割り当てられた状態
…ですが何か？

よくわからん

覚えておいて
ほしい用語

型付け (Typing)

型付き (Typed)

型チェッカー

```
$a = 3;  
$b = 2;  
$c = pow($a, $b);
```

型チェッカー

int

```
$a = 3;  
$b = 2;  
$c = pow($a, $b);
```

型チェッカー

int

int

```
$a = 3;  
$b = 2;  
$c = pow($a, $b);
```

型チェッカー

int

int

```
$a = 3;  
$b = 2;  
$c = pow($a, $b);
```

引数ok

型チェッカー

int

int

```
$a = 3;  
$b = 2;  
$c = pow($a, $b);
```

引数ok

引数ok

型チェッカー

int

int

```
$a = 3;
```

```
$b = 2;
```

```
$c = pow($a, $b);
```

int

引数ok

引数ok

型付け(typing) vs 型付き(typed)

- プログラムを検査して、項($\doteq$ 式, expression)のとりうる状態を分類するのが型付け(typing)。型を明示しなくても項の種類を特定する具体的な手続きが型推論(type inference)。
- 実行時に値の型チェックを行なうことを、便宜的に動的型付け(dynamic typing)と呼ぶこともある
- 静的な型付けによってとりうる状態が定まったプログラムのことを「型付き(typed)」と呼ぶ

1. PHPは型なし言語である
2. PHPには型宣言がある
3. PHPに書く型は実行時に保証されない
4. PHPは静的型付きである

健全性 (Soundness)

静的解析が通れば
実行時に(型に関する)
エラーは起こらない
という性質

いわゆる「型安全」 (type safety)

- 健全性(Soundness)と同じ意味
- 濫用されがちな用語だが、概ね型に関する記述に矛盾がないことを保証できるようにになっていることを表わしている
- 「健全性がある」実行したときに、型に関するエラーが起こらないといえる
- 「健全性がない」実行してみたときに、エラーが起こりかねない
 - C言語全般、Javaの「ぬるぽ」、TypeScriptのany/asや配列など

言語が静的か動的かとは[独自研究]

- ある言語の仕様または標準的な処理系によって構造が事前に検証されていることが前提になっている言語が静的(型付き)
- 事前検証が前提ではなく、実行時(run-time)に構造が決定されるのが動的
 - 一般的にはコンパイラ vs インタプリタに一致するが、必ずしもそうではない
- 静的言語にも多かれ少なかれ動的な機能が存在し、グラデーションは大きい
- 動的言語でもPHP, Python, Ruby, JavaScriptで動的要素が全然違う

TypeScriptについての型

- 型宣言をもとに型チェックを行なって、分析情報を報告
- 実行時には型情報は何も残らない
 - JavaScriptにコンパイル ÷ 型定義の部分をひっぺがして実行
 - Node.js 25.2ではType Strippingで直接実行できるようになった
 - 実際にはTypeScript enum など別のコード生成をする機能もある
- 型がついたコードは実行時にはエラーが出ないでほしい(願望)

ファクトチェック

1. PHPは型なし言語である
2. PHPには型宣言がある
3. PHPに書く型は実行時に保証されない
4. PHPは静的型付きである

型理論から見たPHPインタプリタ

- 型が合わない(Ill-typed)コードであっても、エラーとして弾かれず、正当なプログラム(構文木)として受理される → **実行時エラー**
- 関数は型宣言に拘らず、任意の値を返そうとすることができる
- プロパティは型宣言に拘らず、任意の値を代入しようとすることができる
- ローカル変数に型宣言はなく、任意の型の値を代入可能
- オペコードコンパイラは実行前にプログラムの健全性を「証明」しない

実行時のインタプリタの振る舞い

- PHP処理系(ZendVM)内では、zval構造体に型タグを保持している
 - 常に `is_int()` のような関数で型を識別できる
 - 型宣言と反した値が渡されると、キャストまたは実行時エラーが起こる
- いわゆる「強い型付け」であり、実行時に型がとり換えられることはない
- RTTI(実行時型情報)は型理論における(静的な)型ではない
- 言語セマンティクスとしては型なしだが、JITは静的型情報を活用する

ここまでのまとめ

PHPインタプリタは
型なしである

1. PHPは型なし言語である
2. PHPには型宣言がある
3. PHPに書く型は実行時に保証されない
4. PHPは静的型付きである

PHPと型宣言

- 引数などのデータ型を表わす手段としてコメントにPHPDocを書く文化
- PHP 5.xで関数／メソッドのパラメータに「型ヒント」が書けるようになった
 - array, callable, クラス名, インターフェイス名だけを書けた
- PHP 7.0で正式に「型宣言」としてスカラー型宣言と戻り値型が記述可能に
- PHP 8.0以降でユニオン型宣言などが機能追加

型宣言がない関数(PHP5)

```
function add($a, $b) {  
    return $a + $b;  
}
```

スカラー型宣言(PHP7)

```
function add(int $a, int $b): int {  
    return $a + $b;  
}
```

スカラー型宣言(PHP7)

```
function add(int $a, int $b): int {  
    return $a + $b;  
}
```

int + intって
本当にintなの？

広い値をとるにはfloatが必要

```
function add(float $a, float $b): float {  
    return $a + $b;  
}
```

広い値をとるにはfloatが必要

ひとつの解決策ではあるが… 不必要にfloatを強制するのか

```
function add(float $a, float $b): float {  
    return $a + $b;  
}
```

PHPDocの型注釈(アノテーション)

```
/**
 * @param int|float $a
 * @param int|float $b
 * @return int|float
 */
function add($a, $b) {
    return $a + $b;
}
```

PHPDocの型注釈(アノテーション)

```
/**
 * @param int|float $a
 * @param int|float $b
 * @return int|float
 */
function add($a, $b) {
    return $a + $b;
}
```

あえて型宣言を省略する

ユニオン型宣言 (PHP8.0)

```
function add(int|float $a, int|float $b): int|float {  
    return $a + $b;  
}
```

PHPの型宣言の特徴

- コードごとに `declare(strict_types=1);` が宣言できる
 - 引数とデータ型の種類を厳密にチェックして `TypeError` を出す
- デフォルト(無宣言)では、`add("1", "2")` を `add(1, 2)` としてキャスト
 - `add("a", "b")` のような数値ではない文字列は `TypeError` になり安心
 - `add((int) $a, (int) $b);` のようなコードとは安全性のレベルが違う

ここまでのまとめ

PHPに型宣言はある

型宣言された型は
□約束ではなく
実効性がある

型理論としては型なしだが
がんばって動作確認するなら
実用上ほぼ型付きと
認められなくもなってきた

1. PHPは型なし言語である
2. PHPには型宣言がある
3. PHPに書く型は実行時に保証されない
4. PHPは静的型付きである

PHPDocに 型を書く文化もある

PHPDocの型注釈(アノテーション)

```
/**
 * @param int|float $a
 * @param int|float $b
 * @return int|float
 */
function add($a, $b) {
    return $a + $b;
}
```

PHPDocの型注釈(アノテーション)

```
/**  
 * @param int|float $a  
 * @param int|float $b  
 * @return int|float  
 */  
function add($a, $b) {  
    return $a + $b;  
}
```

あえて型宣言を省略する

いろんなところから湧き出す 型の特定できないデータ

```
1 <?php
2
3 // JSONデータのデコード結果
4 $data = json_decode($str);
5
6 // eval (PHPコードの動的実行)
7 $data = eval($code);
8
9 // DBのクエリ結果
10 $data = $db->query($sql);
11
```

arrayは無限の可能性を秘める (悪い意味で)

```
1 <?php
2
3 function searchUser(int $user_id): array
4 {
5     // ...
6 }
7
8 $user = searchUser(11);
```

 10 \$user[???] ← どんなキーで格納されてるのか不明

11

扱う対象が複数になったのに 型に反映されない

```
1 <?php
2
3 function searchUsers(array $user_ids): array
4 {
5     // ...
6 }
7
8 $users = searchUsers([11, 22, 33]);
9
```

```
 10 $users[???] ← どんなキーで格納されてるのか不明
```

```
11
```

型宣言にarrayとしか書かないと

- メソッドに何を渡せばいいのかわからない
- どんな形式の配列を渡しても動くという関数は決して多くない
 - 標準関数の `count()` のようなユーティリティが多め
- プロパティにも何が入ってるのかわからない
 - 従来のLaravelは `@var array` まみれだった
 - Laravel 9.xのコードベースでは配列の内部構造に型がついている

配列はひとつ！じゃない！！

- ユーザIDがならんだリスト [123, 456]
- ユーザIDと名前の対応表 [123 => '野比のび太', 234 => '源静香']
- ユーザを表す構造体 ['id' => 123, 'name' => '野比のび太']
- ユーザを表す構造体がならんだリスト
[
 ['id' => 123, 'name' => '野比のび太'],
 ['id' => 234, 'name' => '源静香'],
]

配列はひとつ！じゃない！

list<int>

- ユーザIDがならんだリスト [123, 456]
- ユーザIDと名前の対応表 [123 => '野比のび太', 234 => '源静香']
- ユーザを表す構造体 ['id' => 123, 'name' => '野比のび太']
- ユーザを表す構造体がならんだリスト
[
 ['id' => 123, 'name' => '野比のび太'],
 ['id' => 234, 'name' => '源静香'],
]

配列はひとつ！じゃない！

list<int>

array<int,string>

- ユーザIDがならんだリスト [123, 456]
- ユーザIDと名前の対応表 [123 => '野比のび太', 234 => '源静香']
- ユーザを表す構造体 ['id' => 123, 'name' => '野比のび太']
- ユーザを表す構造体がならんだリスト
[
 ['id' => 123, 'name' => '野比のび太'],
 ['id' => 234, 'name' => '源静香'],
]

配列はひとつ！じゃない！！

list<int>

array<int,string>

- ユーザIDがならんだリスト [123, 456]
- ユーザIDと名前の対応表 [123 => '野比のび太', 234 => '源静香']
- ユーザを表す構造体 ['id' => 123, 'name' => '野比のび太']
- ユーザを表す構造体がならんだリスト
[
 ['id' => 123, 'name' => '野比のび太'],
 ['id' => 234, 'name' => '源静香'],
]

array{id:int, name:string}

配列はひとつ！じゃない！！

list<int>

array<int,string>

- ユーザIDがならんだリスト [123, 456]
- ユーザIDと名前の対応表 [123 => '野比のび太', 234 => '源静香']
- ユーザを表す構造体 ['id' => 123, 'name' => '野比のび太']
- ユーザを表す構造体がならんだリスト
[
 ['id' => 123, 'name' => '野比のび太'],
 ['id' => 234, 'name' => '源静香'],
]

array{id:int, name:string}

list<array{id:int, name:string}>

1. PHPは型なし言語である
2. PHPには型宣言がある
3. PHPに書く型は実行時に保証されない
4. PHPは静的型付きである

PHPの組み込みの
型機能ではどんなに
型推論しても
静的に型が定まらない

無限の可能性を秘めたarray
を放置するとコード全体の
エントロピーが増大し
プロジェクトは熱的死を迎える

漸進的型付け

PHPStanは比較により**型を絞り込む**(type narrowing)ことができます。コードに以下のようなコードを追加して型を確認してみてください。

```
$word = filter_var($_GET['word'] ?? '');  
\PHPStan\dumpType($word); // DumpedType: string|false  
  
if ($word === '' || $word === false) {  
    \PHPStan\dumpType($word); // DumpedType: ''|false  
} else {  
    \PHPStan\dumpType($word); // DumpedType: non-empty-string  
}  
  
\PHPStan\dumpType($word); // DumpedType: string|false
```



型チェックをしたとしても
実用的な推論結果にならない
(こともある)

```
$a = 'foo';  
$b = rand() === 1 ? 'foo' : 'bar';  
$c = $a + $b;
```

PHPStan

'foo'

```
$a = 'foo';  
$b = rand() === 1 ? 'foo' : 'bar';  
$c = $a + $b;
```

PHPStan

'foo'

'foo'

```
$a = 'foo';  
$b = rand() === 1 ? 'foo' : 'bar';  
$c = $a + $b;
```

PHPStan

'foo'

'foo'

'bar'

```
$a = 'foo';  
$b = rand() === 1 ? 'foo' : 'bar';  
$c = $a + $b;
```

PHPStan

'foo'

'foo'

'bar'

```
$a = 'foo';
```

```
$b = rand() === 1 ? 'foo' : 'bar';
```

```
$c = $a + $b;
```

PHPStan

'foo'

'foo'

'bar'

```
$a = 'foo';
```

```
$b = rand() === 1 ? 'foo' : 'bar';
```

```
$c = $a + $b;
```

'foo'|'bar'

PHPStan

'foo'

'foo'

'bar'

```
$a = 'foo';
```

```
$b = rand() === 1 ? 'foo' : 'bar';
```

```
$c = $a + $b;
```

'foo'|'bar'

'foofoo'|'foobar'

PHPStan

```
$a = 1;  
$b = 2;  
$c = $a + $b;
```

PHPStan

int(1)

```
$a = 1;  
$b = 2;  
$c = $a + $b;
```

PHPStan

int(1)

int(2)

```
$a = 1;  
$b = 2;  
$c = $a + $b;
```

PHPStan

int(1)

int(2)

int(3)

```
$a = 1;  
$b = 2;  
$c = $a + $b;
```

型チェッカー

```
$json = $_POST['json'];  
$a = json_decode($json);  
var_dump($a['name']);
```

型チェッカー

mixed

```
$json = $_POST['json'];  
$a = json_decode($json);  
var_dump($a['name']);
```

型チェッカー

mixed

string欲しいんだけど...

```
$json = $_POST['json'];  
$a = json_decode($json);  
var_dump($a['name']);
```

型チェッカー

mixed

string欲しいんだけど...

```
$json = $_POST['json'];  
$a = json_decode($json);  
var_dump($a['name']);
```

?array|int|float|string|bool

型チェッカー

mixed

string欲しいんだけど...

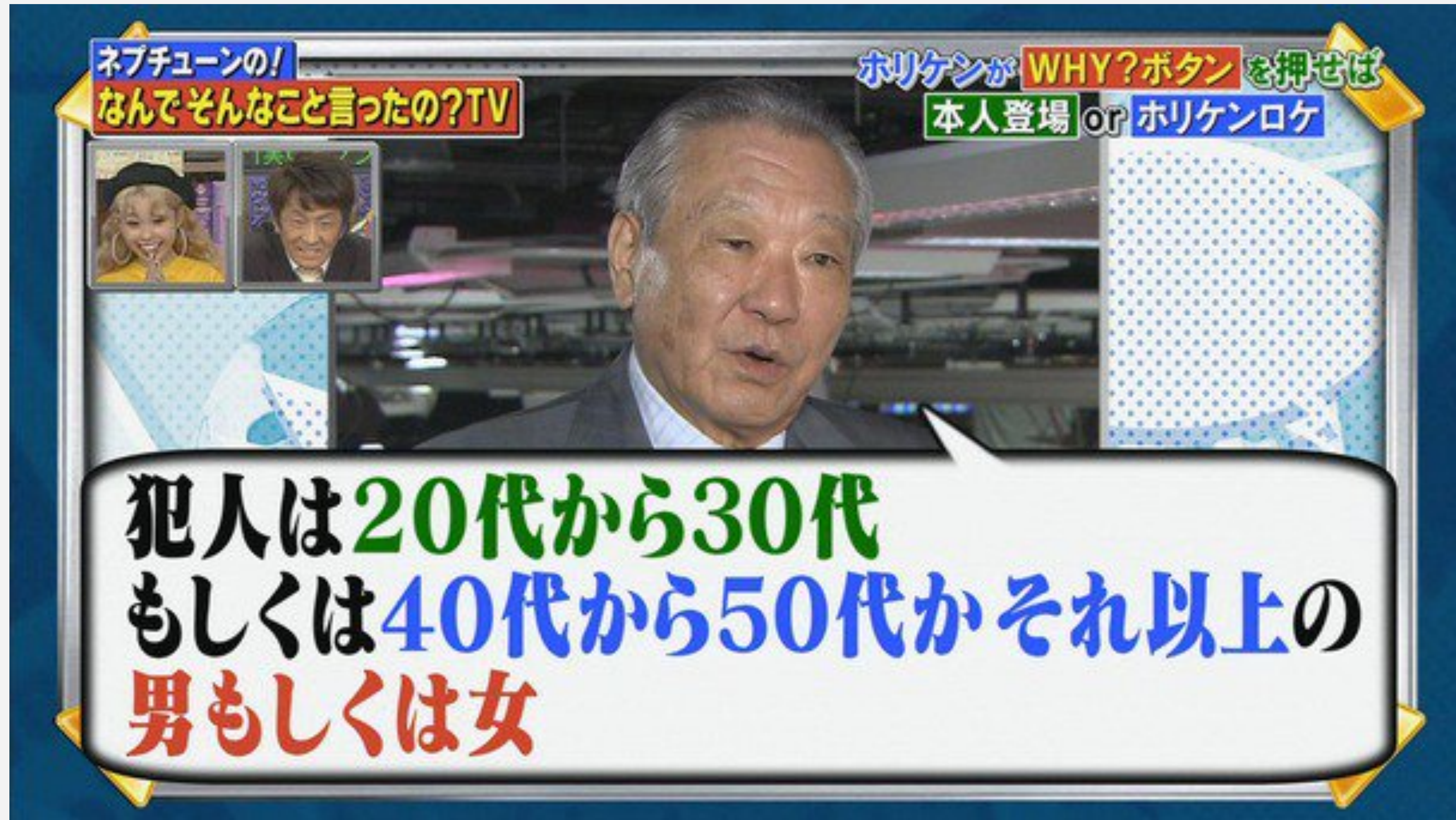
```
$json = $_POST['json'];  
$a = json_decode($json);  
var_dump($a['name']);
```

?array|int|float|string|bool

このキーあるの...?

つまり

つまり



そのため動的言語では
テストの重要性が
高かった

型は「ありうる」
状態を記述する言語

適切に推論できれば
テストを書くまでもなく
変なコードが炙りだせる



PHPStanクイックガイド2023

2023/09/17に公開 2024/12/13



83



10



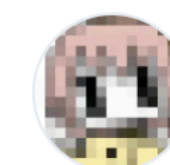
PHPStan（PHP Static Analysis Tool）はコードを実行せずに検査できるツールです。本稿では業務アプリケーションにPHPStanを導入するまでに押さえておきたい事柄を記述します。

! この記事の初出は PHPerKaigi 2023 のパンフレット原稿です。

導入

PHPStanは本稿記述時点の1.9.x系において、PHP 7.2以降で実行できます。PHPStanは `composer require --dev phpstan/phpstan` でのインストールが基本です。

プロジェクトルートの `phpstan.dist.neon` に、以下のように記述してください。



にゃんだーすわん

フォロー



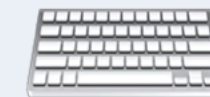
にゃーん

バッジを贈る

バッジを贈るとは →

目次

- 導入
- ・ IDEの有効化
- PHPと型
- 型付けの基礎
- ・ 型宣言
- ・ PHPDocの書き方



PHPStan型付けマニュアル

2025/03/30に公開 🔄 2025/04/15



46



12



PHP



PHPStan



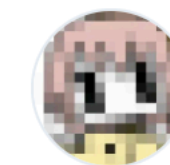
Tech

こんにちは！楽しくPHPStanを使っていますか？それともPHPStanに使われていますか？

PHPStanは非常に賢く、容易にPHPコードの「嫌な気配」を検知してくれます。ただ、PHPStanが指摘することが常に正しいなどといったことはなく、いつでもプログラムが動作しているという事実が前提で、静的解析はその影を追っているに過ぎません。

PHPStanがどのようなメカニズムで型を付けているのか理解できていないと、自我を失って機械に言われるがままに意図しないコードを書かされる人形になってしまいます。本稿ではPHPStanを自律的に使うために前提となる知識を紹介します。

ここからプログラミング言語と型、そしてPHPについての議論を始めたいのですが、「PHP」という名前はプログラミング言語の名前であり、PHP言語で書かれたソースコードを実行するプログラ



にゃんだーすわん

フォロー



にゃーん

バッジを贈る

バッジを贈るとは →

目次

- 型システムって何？
- PHPの組み込み型
- 型宣言と型モード
- 「型が付く」とはどういうことか

PHPStan型付けチュートリアル

PHPStanを使った型付けをWebブラウザ上で体験できるチュートリアルです。

コード編集中にリアルタイムでPHPStanのフィードバックを得て「**PHPStanとペアプログラミングをする**」感覚を獲得してください。

チュートリアル

1. 🌱 [PHPStan型付けチュートリアル 入門編](#)
2. 📖 [PHPStan型付けチュートリアル 基礎編 \(工事中\)](#)

取り組み方

チュートリアルには節見出しごとに、以下のようなブロックがあります。

📘 Note

この節のコードは以下で確認できます

- **PHPStan Playground:** `https://phpstan.org/...`
- **File:** `xxx.php`
- **CLI:** `./vendor/bin/phpstan analyze beginner/xxx.php`

パラメータを
厳しくするより
戻り値を適切に

戻り値が適切に
記述されていれば
呼び出し側で
型をつける必要はない

PHPStanは
普通にコードを書いて
警告が増加しない
レベルが適正

複雑な型を
書きまくるのが
型付けではない

PHPコードの
@returnが
足りていない

💡 型を書きまくる
SKILLを書こう

特にarray-shapeを
書きまくる
必要が生じる

そこからレベルを
上げるには
拡張で解決したい

たぶんあんまり
よくない

型推論ロジックを記述できる

//Zenn



投稿する



pixiv

Publicationへの投稿

Edit



キミにも作れるPHPStan拡張

2024/04/13に公開

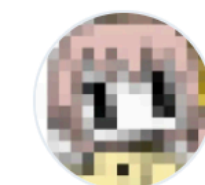
PHP

PHPStan

Tech

こんにちは！ PHPStanを活用していらっしゃいますでしょうか。

PHPStanは今日では言わずと知れた静的解析ツールですが、プラグイン機構を備えており、型付けのためのさまざまな機能を提供しています。PHPStanはできるだけコードを書かずPHPDocによる記述だけで多くのケースには型をつけられるようになっていますが、残念ながら本稿執筆時点のPHPStan 1.10系ではPHPDocだけですべてのニーズを満たすことはできないため、より深くPHPStanを利用するにはプラグインの理解が必要になります。



にゃんだーすわん



にゃーん

バッジを贈る

バッジを贈るとは →

目次

理屈上は任意の項に
型をつけられる

お使いのAIさんと
相談しながらサクッと
作れる時代になった

やるなら今

LLMにとって
プログラミング言語と
人間の言語は同列

流暢なコードは
効率よく処理しやすい

エントロピーを下げて
人とAIに優しい
予測可能なコードを
育てよう

まとめ

正しいものは？（複数回答可能）

1. PHPは型なし言語である
2. PHPには型宣言がある
3. PHPに書く型は実行時に保証されない
4. PHPは静的型付きである

正しいものは？（複数回答可能）

- ✓ PHPは型なし言語である
- ✓ PHPには型宣言がある
- ✓ PHPに書く型は実行時に保証されない
- ✓ PHPは静的型付きである

考え方によつては
すべて正しい

PHPが静的型付きと言える条件

- 静的解析やユニットテストなどのCIチェックした上でデプロイされている
 - PHPStan, Psalm, Phan, Magoなど… Lv.maxかどうかは問わない
 - ユニットテストで十分なカバレッジのテストが網羅されている
- ツールの種類×プロジェクトが静的に信頼に足る状況にできるかが大きい
 - 静的型付きの言語を使っても本番で健全性がないコードにできる
 - PHPは動的言語の中では実行時バリデーションのお蔭で問題が隠れない

やわPHP カタPHP

AI Dente PHP: Finding the Perfect Resilience



pixiv Inc.
USAMI Kenta

pixiv

1

2026-05-09 #phpconkagawa
香川県高松市 玉藻公園内 披雲閣